

Is Agentic AI Ready for Real-World Hardware Engineering? A Deep Dive with Phoenix-bench

Qingyun Zou¹ Feng Yu¹ Hongshi Tan¹
 Jiahao Cui¹ Bingsheng He¹ Weng-Fai Wong¹
¹National University of Singapore

Abstract

We ask whether agentic AI systems built for software engineering transfer to realistic hardware engineering. Existing hardware LLM benchmarks isolate sub-tasks but none jointly requires repository navigation, hierarchy-aware localization, Electronic Design Automation (EDA) executable verification, and maintenance-style patching. We introduce **Phoenix-bench**, a synchronized corpus of 511 verified Verilator instances from 114 GitHub repositories, each shipped with the developer patch, design-flow labels, fail-to-pass and pass-to-pass testbenches, and a Docker-pinned EDA environment so resolved-rate differences reflect agent behavior rather than toolchain availability. Using Phoenix-bench we run a uniform evaluation of four commercial agents and eight open-source agentic structures across four LLM backbones, plus two diagnostic interventions (file-level oracle localization and one round of testbench-log feedback). Three findings emerge. (i) Software and hardware are fundamentally different engineering tasks: the same agent loses 37% to 58% from SWE-bench Verified to Phoenix-bench because hardware bugs propagate across parallel instantiated modules through signal flow rather than along a software-style call graph, and software-tuned agents stop at the symptom file instead of tracing back through the instantiation chain. (ii) Failures concentrate on design control-flow / finite state machine (FSM) bugs, verification testbench bugs, and hard cases that demand cross-hierarchy signal-flow tracking and coordinated multi-file edits. (iii) Localization granularity matters far more than localization itself: a perfect file-level oracle yields only +1.4% because the agent then breaks files that did not need editing, while a single round of test case feedback lifts resolved rate by 42% to 45% because the test case tells *where* the bug is and *what* the fix has to look like.

1 Introduction

Agentic AI has been remarkably successful on software engineering tasks, where coding agents such as SWE-Agent (Yang et al., 2024), OpenHands (Wang et al., 2024), mini-SWE-agent (SWE-agent Team, 2025), and Agentless (Xia et al., 2024) now capable of resolving a large fraction of real GitHub software repository issues end-to-end on benchmarks like SWE-bench (Jimenez et al., 2023). Despite the use of high-level programming languages, hardware design is fundamentally different: a hardware design is a circuit, i.e., a parallel network of instantiated modules wired together by signal flow, rather than a sequential program executed along a call graph. Whether agents tuned for the call-graph world of software transfer to this parallel, signal-flow world of hardware is an open question. Resolving repository-level hardware issues raises a concrete question: will the same software coding agent be able to obtain hardware designs expressed in high-level languages and pass EDA tests, and, if not, what more is required?

Existing hardware LLM benchmarks isolate useful capabilities but do not yet cover the full hardware design workflow. Verilog generation benchmarks test whether a model can write standalone modules (Thakur et al., 2024; Liu et al., 2023; Lu et al., 2024); repair and debugging benchmarks

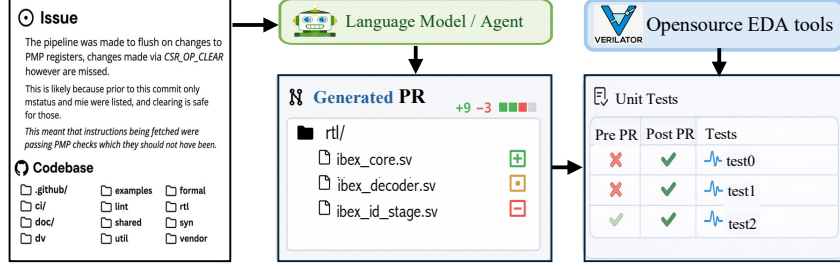


Figure 1: Phoenix-bench task overview: an agent edits a Verilog/SystemVerilog repository in response to a real GitHub issue and is judged by Docker-based EDA fail-to-pass and pass-to-pass tests.

test localized fixes (Tsai et al., 2024; Yao et al., 2025a; Ahmad et al., 2023); other work studies bug localization (Yao et al., 2025b; Zhou et al., 2025; Fang et al., 2024). These are valuable, but a project-wide full intervention requires repository navigation, hierarchy-aware localization, executable verification, and maintenance-style patching. It is unclear if existing software coding agents can perform this end-to-end.

We introduce **Phoenix-bench**. As shown in Figure 1, a benchmark in Phoenix-bench comprises a real GitHub issue and a full Verilog/SystemVerilog repository, and edits the repository so that the edited version passes both deterministic fail-to-pass and pass-to-pass Verilator tests. The synchronized corpus contains 511 verified instances from 114 GitHub repositories, selected from a crawl of 18,010 issues across 786 repositories, each shipped with developer patch, design-flow labels, and a Docker-pinned EDA environment so that resolved-rate differences reflect the agent’s behavior rather than the availability of the tool.

Using Phoenix-bench, we did a comprehensive evaluation of four commercial coding agents and eight open-source agentic structures across four LLM backbones, along with two diagnostic interventions (file-level oracle localization and one round of testbench-log feedback). Three key findings emerge from our study: (i) Software and hardware are fundamentally different engineering tasks, and software coding agents do not perform well. For example, OpenHands with Qwen3-Coder-480B went from 69.6% accuracy on SWE-bench Verified to 32.3% on Phoenix-bench. (ii) The agents’ failures concentrated in specific sets of issues not found in software, and on a handful of particularly difficult cases. (iii) Localization granularity matters more than localization itself. A perfect file-level oracle yields only 1.4% improvement because the agent does not know what kinds of hardware bugs (synthesis, functions or others) and ignore files that were identified.

The contributions of this paper are:

- **Phoenix-bench, a hardware-issue-resolution benchmark.** A synchronized corpus of 511 verified Verilator instances from 114 GitHub repositories, paired with developer patches, design-flow labels, fail-to-pass and pass-to-pass testbenches, and Docker-pinned EDA images so the resolved-rate metric reflects agent behavior rather than toolchain availability.
- **Comprehensive evaluation of commercial and open-source agents.** Four state-of-the-art commercial software coding agents (Claude Code, OpenAI Codex, Gemini CLI, GitHub Copilot) and eight open-source agentic structures across four LLM backbones, plus two diagnostic interventions (file-level oracle localization and testbench-log feedback) were tested using Phoenix-bench.
- **Actionable hardware-specific findings for future AI approaches.** An analysis of the evaluation results yielded insights that we believe will improve the design of future agents for hardware design.

2 Related Work: Hardware Code Benchmarks

Existing hardware code benchmarks are usually described by visible attributes such as module versus repository, generation versus repair, and exact-match versus simulation. These attributes are useful but insufficient: they do not explain why a benchmark reflects, or fails to reflect, real hardware engineering. We instead ask whether a benchmark captures repository context, cross-module signal dependencies, execution-grounded EDA verification, maintenance-style patching, and issue diversity across the hardware design flow. Table 1 separates hardware benchmarks by the engineering behavior they require. Module-level benchmarks such as VerilogEval (Liu et al., 2023), RTLML (Lu et al.,

Table 1: Coverage of hardware engineering practice across existing hardware benchmarks. Part. denotes partial coverage.

Benchmark	Engineering setting	Task	Verification	Design-flow coverage	Testbench coverage
VerilogEval (Liu et al., 2023)	Isolated module	Code generation	EDA simulation	Design only	No
RTLLM (Lu et al., 2024)	Isolated module	Code generation	EDA simulation	Design only	No
RTLFixer (Tsai et al., 2024)	Isolated module	Syntax repair	Compilation	Design only	No
HDLdebugger (Yao et al., 2025a)	Isolated module	Debug repair	Simulation	Design/verification	No
RTL-Repo	Repository snippets	Completion	Exact match	Design only	No
MHRC-Bench (Zou et al., 2026)	Repository context	Completion	Exact match	Design only	No
HWFixBench (Fu et al., 2025)	Repository repair	Patch generation	LLM judge	Limited	No
Phoenix-bench	Real issue + repo	Bug repair	EDA execution	Different stages	Yes

2024), HDLEval (Kashanaki et al., 2024), PyHDL-Eval (Batten et al., 2024), RTLFixer (Tsai et al., 2024), and HDLdebugger (Yao et al., 2025a) test generation, repair, or debugging in restricted contexts, so they do not require cross-file consistency or hierarchy-level signal tracing. Repository-level completion benchmarks such as RTL-Repo and MHRC-Bench (Zou et al., 2026) add larger context, but exact-match completion does not model issue resolution. HWFixBench (Fu et al., 2025) moves closer to repository repair; Phoenix-bench differs by pairing real issue reports with localization-to-patch behavior and executable EDA verification.

3 Phoenix-bench Benchmark Suite

This section describes the Phoenix-bench workload, including data collection, issue categories, artifact statistics, and verification.

Design principles. Phoenix-bench uses five design principles to keep the task close to hardware maintenance. First, each issue is resolved in the full repository, because the relevant behavior may span instantiated modules, packages, generated artifacts, and build scripts. Second, localization must be able to follow hierarchical signal propagation instead of relying only on lexical overlap. Third, correctness is measured by simulator, compiler, synthesis, or exact artifact checks. Fourth, issue categories cover design, verification, toolchain, implementation, and documentation artifacts. Fifth, the output is a repository patch, not standalone RTL.

Data collection. Phoenix-bench is built through the four-stage funnel in Figure 2. We first query the GitHub Search API for Verilog/SystemVerilog repositories with at least 50 stars, then retain issues linked to pull requests through explicit references or GitHub native linking. We next filter for pull requests that modify Verilog/SystemVerilog artifacts and remove duplicate or incomplete cases. This process yields a synchronized runner corpus of 511 verified Verilator instances with both fail-to-pass and pass-to-pass tests; analyses that require manually curated design-flow labels use the labeled subset, while runner, repository, and coverage statistics use the full corpus.

Design-flow categories. Phoenix-bench labels issues by the hardware artifact whose behavior must change. *Design* issues cover RTL behavior such as functional bugs, specification mismatches, and quality-of-results concerns. *Verification* issues cover testbench, assertion, or simulation-build failures. *Environment & Toolchain* issues cover simulator compatibility, build scripts, and tool invocation. *Physical & Implementation* issues cover synthesis-stage effects such as timing warnings, latch inference, and resource use. *Documentation* issues cover README files, comments, and API descriptions. We stop at synthesis because backend stages such as place-and-route and clock-tree synthesis are not deterministically reproducible across the collected repositories in our current environment.

Dataset composition. The collected issues are not distributed like the final benchmark. Documentation issues dominate the raw crawl, but code-linked issues shift toward toolchain and design changes because those categories more often produce concrete pull-request patches. The labeled subset in Figure 3(b) keeps all five categories while over-sampling design issues, which are the main setting for testing RTL semantic repair. The full synchronized corpus is broader: its 511 instances span 114 GitHub repositories, including processor cores, common-cell and protocol-IP libraries, FPGA/SoC applications, and simulator/toolchain repositories. Table 2 reports testcase volume and patch-local

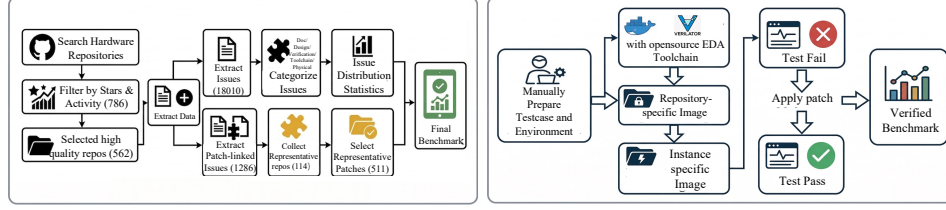


Figure 2: Phoenix-bench construction pipeline, from GitHub crawl to verified Docker-based instances.

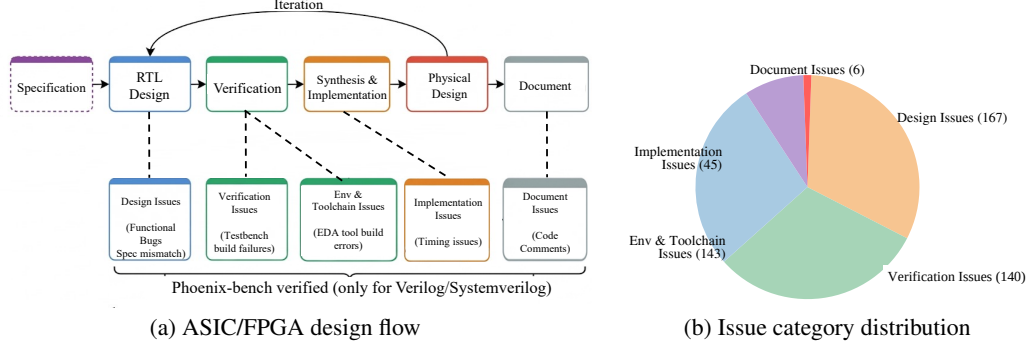


Figure 3: ASIC/FPGA design flow mapped to Phoenix-bench issue categories (left) and the 511-instance issue-category distribution (right).

coverage rather than only runner counts. The harness emits 71,048 TESTCASE: checks across the synchronized corpus. Patch coverage is computed over patch-touched executable lines, Verilator branch/toggle points, and recovered patch-local function or scope entries, which keeps the coverage metric tied to the code changed by the developer patch.

Verification environment. Each instance is verified in a three-layer Docker environment built on a portfolio of open-source EDA tools: Verilator for cycle-accurate simulation, Icarus Verilog as an alternative event-driven simulator for projects whose testbenches use SystemVerilog constructs Verilator does not support, Surelog for SystemVerilog parsing and elaboration, and Yosys for synthesis and lint-style checks. The base image provides this toolchain, a repository image installs project dependencies, and an instance image pins the issue snapshot. Two execution checks gate acceptance: a *fail-to-pass* check requires the targeted testbench to fail on the buggy snapshot and pass after applying the developer patch, and a *pass-to-pass* check requires the agent’s patch to keep all previously-passing testbenches still passing so the patch is not allowed to break unrelated functionality. All 511 instances are verified through this open-source EDA toolchain, and the acceptance criterion for all headline results is execution-grounded verification, not LLM judgment.

4 Experiments: Product Agents, Open-source Agents, and Diagnostics

4.1 Evaluation protocol

We use Phoenix-bench to study three questions. **RQ1:** How well do real product coding agents handle repository-level hardware tasks under EDA verification? (§4.2, Table 3) **RQ2:** Which open-source agent structures from software engineering transfer to hardware engineering? (§4.2, Table 4) **RQ3:** Where do current failures concentrate, and which factors, task domain, patch complexity, localization, or testbench feedback, best explain the residual? (decomposed in §5.2 to §5.5)

Evaluation modes. Phoenix-bench separates external validity from mechanism control. The *product-agent* mode evaluates deployed coding agents as black boxes through their native interfaces, preserving their orchestration, model selection, planning, and editing behavior. The *open-source-agent* mode runs reproducible open-source agent structures under the same Docker evaluator and a shared set of LLM backbones, so that model choice, localization strategy, and testbench-feedback handling can be compared directly.

Product agents. We evaluate four state-of-the-art commercial coding agents through their native interfaces, namely Claude Code (Anthropic, 2026), OpenAI Codex (OpenAI, 2026), Gemini CLI (Google,

Figure 4: Repository-composition across the 114 GitHub repositories used in Phoenix-bench.

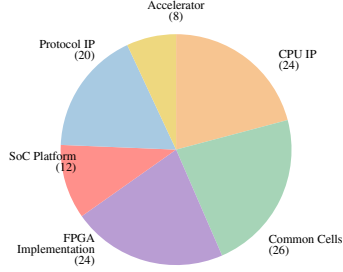


Table 2: Benchmark statistics: SWE-bench Verified (500 Python instances) versus the current Phoenix-bench synchronized runner corpus (511 Verilog/SystemVerilog instances).

		SWE-bench		Phoenix-bench	
		Mean	Max	Mean/Agg.	Max
Issue Text	Length (words)	195.1	4,477	84.4	2,104
Codebase	# Files	3,010	5,890	1,153.2	13,061
	# Lines	438K	886K	132.7K	2.08M
Gold Patch	# Lines edited (HDL)	32.8	5,888	737.5	113,849
	# Files edited (HDL)	1.7	31	5.7	368
	# Patch scopes edited	3.0	36	8.4	348
Tests	# Fail-to-pass tests/checks	9.1	1,633	3.5	82
	# TB Lines	120.8	9,459	49.8	219
	Patch Line Coverage (%)	–	–	99.91	100
	Patch Branch Coverage (%)	–	–	100.0	100
	Patch Toggle Coverage (%)	–	–	100.0	100
	Patch Function Coverage (%)	–	–	100.0	100

2026), and GitHub Copilot coding agent (GitHub, 2026). These systems are evaluated as black boxes because their engineering value lies in the complete product loop, not only in the underlying model. Each product receives the same issue text, repository snapshot, Docker verifier, time budget, and final acceptance gate. We record the generated patch, transcript or action log when available, EDA output, wall-clock time, test-call count, and token or credit cost when the product exposes it. **Open-source agents.** We additionally instantiate or adapt eight representative agentic structures under Phoenix-bench in three categories. *Agent-based*: mini-SWE-agent (SWE-agent Team, 2025), SWE-agent (Yang et al., 2024), OpenHands (Wang et al., 2024), Aider (Gauthier, 2024), KGCompass (Yang et al., 2025), and Lingxi (Lingxi Team, 2025). *Retrieval-augmented*: ExpeRepair (Mu et al., 2025). *Workflow-based*: Agentless (Xia et al., 2024). All open-source agents are given the same hardware adaptations where applicable, including EDA toolchain access and SystemVerilog-aware filtering.

Models. Each product agent is evaluated with its default backbone; the specific model used at the time of writing is reported in the Backbone column of Table 3. The open-source agents are evaluated with four LLM backbones spanning proprietary and open-source systems: GPT-5.2 (OpenAI, 2025), Gemini-3-Pro (Google DeepMind, 2025), DeepSeek-V3.2 (Liu et al., 2025), and Qwen3-Coder-480B (Team, 2025). Additional model results can be integrated into the same protocol.

Metrics. *Resolved Rate* is the primary end-to-end metric: a task is resolved only if the generated patch passes the EDA testbench execution. *File Precision* is the fraction of agent-modified files that appear in the developer patch, and *File Recall* is the fraction of developer-patch files modified by the agent. *Module Precision* and *Module Recall* are computed by extracting module/endmodule boundaries from both patches and mapping each modified region to its enclosing module. Product-agent reports additionally include wall-clock time, cost or credit usage when exposed, and EDA-test invocation counts. For benchmark reliability, leaderboard submissions should report headline resolved rates with bootstrap confidence intervals over instances.

4.2 Results

Table 3 reports the head-to-head results for the four product agents on Phoenix-bench. Across resolved rate and localization recall, the agents differ not only in raw success rate but also in how aggressively they invoke the EDA toolchain and in their per-instance wall-clock and cost profile. Agents with deeper testbench-feedback loops resolve more cases at the price of larger token and time budgets, while issue-to-PR style agents finish faster but recover fewer developer-patch files.

Beyond resolved rate, the localization metrics show that file-level coverage saturates earlier than module-level coverage: agents that locate the right files still miss enclosing modules and connected ports, which is consistent with the hardware-specific localization story Phoenix-bench is designed to expose. The cost and EDA-call columns make this trade-off explicit per product, so the table can be read as both an external leaderboard and a profile of how each product spends its budget.

Table 4 shows that software-oriented open-source agents transfer only partially to hardware repositories. Interactive systems such as OpenHands and SWE-agent resolve more cases than static retrieval or workflow systems, but their file-level exploration still misses hierarchy-coupled dependencies. KGCompass and Lingxi perform weakly in this setting, which suggests that software knowledge

Table 3: Product-agent results on Phoenix-bench under the same Docker EDA verifier. **Bold** marks the best value per column.

Product agent	Backbone	Resolved Rate (%)	File Prec. (%)	File Recall (%)	Module Prec. (%)	Module Recall (%)	Tokens (/inst.)
Claude Code	Claude Opus 4.7	38.6	38.6	54.2	47.9	48.3	432k
OpenAI Codex	GPT-5.5	38.0	41.2	52.5	49.7	47.1	326k
Gemini CLI	Gemini-3.1-Pro	37.4	37.4	50.9	46.5	45.4	254k
GitHub Copilot coding agent	GPT-5.3-Codex	32.7	47.8	43.6	48.4	40.2	148k

Table 4: Open-source agentic factor study on Phoenix-bench (%). **Bold/underline** mark the top-1/top-2 values per row.

Model	Metric	Retrieval	Workflow	Agent-based					
		ExpeRepair	Agentless	mini-SWE	SWE-Agent	OpenHands	Aider	KGCompass	Lingxi
GPT-5.2	Resolved Rate	11.7	18.8	14.5	<u>27.8</u>	33.9	16.2	7.2	6.3
	File Precision	<u>52.7</u>	45.8	45.3	60.1	36.2	45.1	32.1	34.3
	File Recall	24.2	37.5	32.2	<u>49.4</u>	50.7	34.6	20.6	22.5
	Module Precision	<u>50.2</u>	37.7	40.1	53.0	46.3	38.6	26.4	28.3
	Module Recall	24.0	32.0	31.1	<u>44.5</u>	45.0	31.1	17.6	19.2
Gemini-3-Pro	Resolved Rate	10.8	13.3	13.3	<u>27.0</u>	33.5	15.3	7.2	6.3
	File Precision	<u>54.3</u>	36.9	42.0	59.5	35.6	44.6	25.8	27.6
	File Recall	19.0	31.1	32.7	<u>49.3</u>	49.8	34.5	17.1	18.7
	Module Precision	51.3	28.7	34.5	<u>50.1</u>	45.4	37.6	20.1	21.6
	Module Recall	18.8	25.1	30.2	<u>43.1</u>	44.2	30.2	13.8	15.1
DeepSeek-V3.2	Resolved Rate	7.2	10.8	8.0	<u>16.2</u>	26.0	9.0	4.5	3.5
	File Precision	57.9	22.5	43.7	<u>48.4</u>	36.6	36.3	15.7	16.9
	File Recall	11.6	19.2	24.2	<u>39.8</u>	49.2	27.8	10.6	11.5
	Module Precision	55.4	17.1	42.1	<u>39.5</u>	<u>46.3</u>	29.6	11.9	12.8
	Module Recall	11.5	15.3	24.2	<u>34.5</u>	42.9	24.2	8.4	9.1
Qwen3-Coder-480B	Resolved Rate	10.0	12.5	12.5	<u>24.3</u>	32.3	13.3	6.3	5.5
	File Precision	59.7	31.5	36.3	<u>53.8</u>	34.3	40.3	22.0	23.6
	File Recall	16.2	25.3	31.3	<u>47.4</u>	49.6	33.2	13.9	15.2
	Module Precision	57.9	25.1	30.5	44.3	<u>45.7</u>	33.2	17.6	18.8
	Module Recall	16.2	21.6	28.9	<u>41.2</u>	42.4	28.9	11.9	13.0

graphs and software-style multi-agent decomposition are not enough to model HDL signal flow or testbench feedback. Hardware-aware localization changes the precision-recall trade-off: conservative systems such as ExpeRepair keep high file precision but miss many developer-patch files, while broad interactive systems recover more files at lower precision. Stronger LLM backbones improve all systems, but the relative ordering between agents remains stable.

Figure 5 contrasts the total token budget consumed by each open-source agent across the 511 Phoenix-bench instances. The two axes of the table, resolved rate and localization recall, are not free: interactive agents that recover more developer-patch files also consume one to two orders of magnitude more tokens. OpenHands sits at the high end (~ 673 M tokens) while retrieval and workflow systems such as KGCompass and Agentless finish the same workload with under 40M tokens. ExpeRepair occupies a middle band, trading some interaction depth for a roughly $6\times$ smaller budget than OpenHands at comparable file precision. This view makes the cost side of the agentic factor study explicit and complements the per-instance token column in Table 3.

5 Key Insights

5.1 How well do software agents fare on hardware design repositories?

Figure 6 contrasts each system’s public SWE-bench Verified score with its Phoenix-bench score for four representative (agent, backbone) pairs (rows in Table 4): all four sit in the 60 to 73% band on SWE-bench yet lose -37 to -58 pp on Phoenix-bench, the strongest SWE-bench pair (mini-SWE-agent + GPT-5.2 at 72.8%) is the weakest on Phoenix-bench (14.5%), and the spread tracks the agent rather than the backbone—mini-SWE-agent drops -52 to -58 pp across GPT-5.2 / Gemini-3-Pro / DeepSeek-V3.2, whereas OpenHands + Qwen3-Coder-480B drops only -37 pp despite a weaker backbone. The headline number also hides two qualitative shifts: SWE-bench failures are

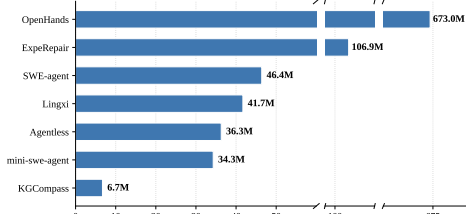


Figure 5: Total token consumption of open-source agents across the 511 Phoenix-bench instances (broken x-axis).

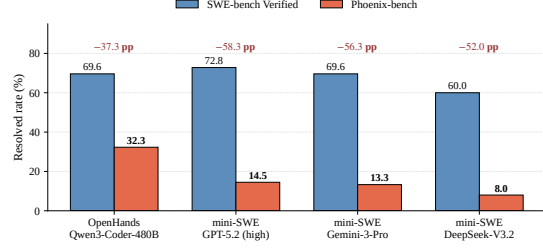


Figure 6: SWE-bench Verified vs Phoenix-bench for four representative (agent, backbone) pairs;

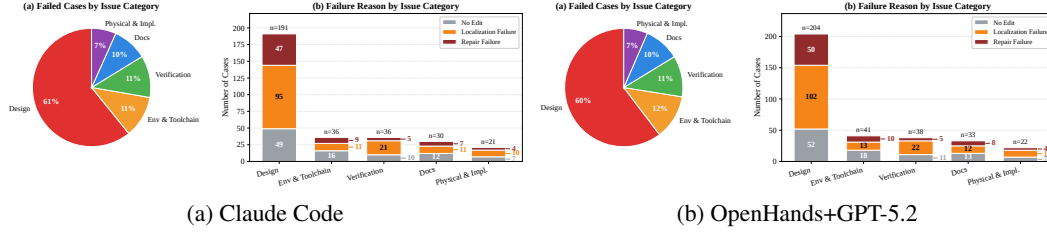


Figure 7: Failure distribution on 511 cases, by issue category (pie) and three-stage taxonomy.

Python-style edge-case logic and missing tests (Jimenez et al., 2023), while Phoenix-bench failures concentrate in hardware-specific categories (port/width, clock/reset polarity, parameter/hierarchy, tool-pragma, procedural-construct misuse), and gold-patch shape changes accordingly—the median SWE-bench patch touches 1 file / 1 hunk / 7 lines with only 14.2% multi-file, vs. the median Phoenix-bench patch at 4 files / 7 hunks / 65 lines with 73.1% multi-file. Interaction depth, not raw backbone strength, is therefore the agent-side property that most reliably survives the SW→HW shift.

Hardware description languages describe circuits as many parallel instantiated modules wired together by signal flow, so fixing a bug typically requires following a signal across instantiations to find the related code. A specific behavioral mismatch in such a setting explains much of the per-agent spread: localization stops at the wrong granularity. On SWE-bench an agent reaches the offending function in one or two file-browse steps, but on Phoenix-bench the same browse pattern finds the file where the symptom appears but does not trace back through the instantiation chain. This surfaces later as the 77% upstream-failure share in §5.2 and the −113 file-oracle regressions in §5.4. It is this issue rather than a capability gap in the underlying LLM that explains the failures.

Main finding: *Software and hardware are fundamentally different engineering tasks: the same agent loses 37 to 58 percentage points from SWE-bench Verified to Phoenix-bench because hardware bugs propagate across parallel instantiated modules through signal flow rather than living in one file along a sequential call graph, so software-tuned agents stop at the symptom file rather than tracing back through the instantiation chain.*

5.2 Where do unresolved failures concentrate?

Figure 7 decomposes every failure on the 511-case corpus along issue category and a three-stage taxonomy where each failure is tagged *No Edit* (0 HDL files touched), *Localization Failure* (HDL files edited but not all gold-patch files covered), or *Repair Failure* (every gold file reached but the fix is wrong). Both Claude Code and OpenHands+GPT-5.2 land in the same 77%/23% upstream/Repair split (29.9% No Edit + 47.1%/47.3% Localization vs. 22.9%/22.8% Repair); failures concentrate in design issues ($\approx 60\%$), with the four other categories each contributing $\leq 12\%$. Figure 8 drills into each issue category and shows that the failure mass is concentrated in a small number of subcategories. Control-flow / FSM bugs produce 152/203 Claude and 162/217 OpenHands design failures (within-subcategory failure rates 63%/68%); testbench bugs produce 32/43 and 34/45 verification failures (74%/79%); Env-and-Toolchain failures spread evenly across compiler/build, simulator-mismatch, and language-standard issues, while Physical-and-Implementation is dominated by clock/reset/CDC and platform-integration. Per-segment within-subcategory failure rates remain in the 40 to 100% band on both panels.

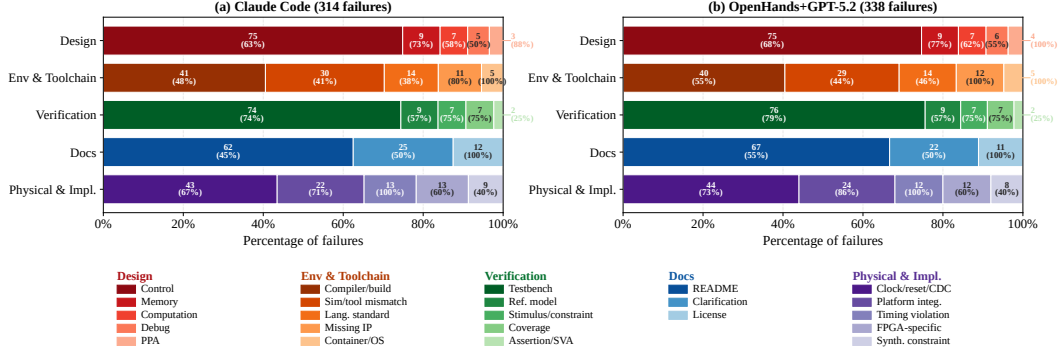


Figure 8: Per-category failure breakdown into fine-grained subcategories, for (a) Claude Code and (b) OpenHands+GPT-5.2. Each row shows the within-category share of each subcategory; the number above each segment is the absolute failure count and the within-subcategory failure rate.

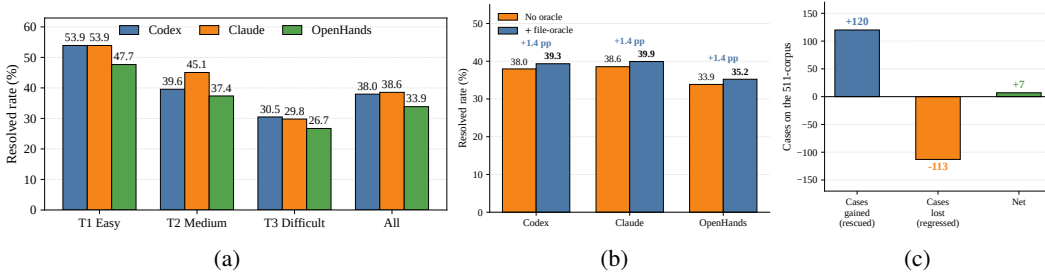


Figure 9: (a) Resolved rate by patch-complexity tier (b) Resolved rate without and with file-level oracle. (c) Per-case decomposition on Claude: rescues vs. regressions.

Main finding: Agents fail mostly on design control-flow / FSM bugs and on verification testbench bugs, with physical failures clustered on clock/reset/CDC and platform integration; env-and-toolchain failures are a long tail of compiler/build, simulator-mismatch, and language-standard issues.

5.3 How does performance vary with patch complexity?

We partition the 511 instances into *T1 Easy* (single hunk, single module), *T2 Medium* (2 to 5 hunks within one file), and *T3 Difficult* (≥ 6 hunks or multi-file or cross-hierarchy edits); the exact rules and a coarser file-and-line-based difficulty distribution are in Appendix A. Figure 9(a) shows that OpenAI Codex, Claude Code, and OpenHands+GPT-5.2 all drop monotonically by roughly $1.8\times$ from T1 to T3 (Codex $53.9 \rightarrow 30.5$, Claude $53.9 \rightarrow 29.8$, OpenHands $47.7 \rightarrow 26.7$); the T3 absolute level (≈ 27 to 31%) sits more than 20 pp below each agent’s own T1. The inter-agent gap stays within ≤ 7 pp at every tier, Codex and Claude tie at T1, Claude leads at T2 (45.1% vs. 39.6%), and Codex edges Claude back at T3 (30.5% vs. 29.8%); Claude’s overall lead (38.6% vs. Codex 38.0% vs. OpenHands 33.9%) is driven by T2.

Main finding: All three agents perform poorly on T3 (hard) cases, landing under one third resolved, because these cases demand cross-hierarchy signal-flow tracking and coordinated multi-file edits that current agents struggle to produce.

5.4 How much can better file localization help?

We give each agent the gold-patch file list at prompt time and otherwise leave the editing pipeline unchanged. Figure 9(b) shows that Claude Code moves only from 38.6% to 40.0% ($\Delta = +1.4$ pp); Codex and OpenHands+GPT-5.2 land within ± 0.1 pp of the same Δ , consistent with the gain and regression mechanisms (panel (c)) being largely backbone-agnostic. Figure 9(c) decomposes Claude’s net +7 cases into +120 rescued failures (the file hint genuinely unblocks the agent) and -113 regressions on baseline-PASS snapshots; four mechanisms explain the offset: a file hint says nothing about which line, operator, or direction to change so the agent’s wrong-guess rate consumes most of the gain; the file list reads as “edit something,” so on the $\sim 47\%$ of testbench-passing

Table 5: Test feedback impact. **(a)** Resolved rate before vs. after one round of feedback for the three interactive agents. **(b)** Rescue rate by Claude’s pre-feedback failure stage on the 314 failures.

(a) Per-agent resolved rate				(b) Claude rescue rate by stage		
Agent	No fb.	+ TB fb.	Δ	Pre-feedback stage	Rescued	Rate
OpenAI Codex	194/511 (38.0%)	419/511 (82.0%)	+44.0	No Edit	73/94	77.7%
Claude Code	197/511 (38.6%)	425/511 (83.2%)	+44.6	Localization Failure	109/148	73.6%
OpenHands+GPT-5.2	173/511 (33.9%)	388/511 (75.9%)	+42.1	Repair Failure	50/72	69.4%

snapshots the agent breaks otherwise-passing builds; a flat list omits the cross-module propagation chain, so the agent fixes the listed file but leaves upstream/downstream files alone; and the oracle cannot express “this file is correctly silent,” so a baseline-PASS check before editing would avoid most regressions.

Main finding: *Telling the agent which file to edit barely helps, because the agent then breaks files that did not need editing; useful localization hints must be finer than the file level.*

5.5 How much does testbench feedback contribute?

Unlike a flat file list, Phoenix-bench’s verifier emits Verilator compile and runtime logs from the buggy phase that go strictly beyond “which file to edit”: they additionally pinpoint the offending line and operator, name the implicated module along its instantiation chain, and supply the expected widths/types/values that the bug violates. We contrast (i) *no feedback*, where the agent receives only issue text and repository, with (ii) *testbench feedback*, where the same agent reads Verilator’s log after each attempt and applies at most one corrective re-edit. Table 5(a) shows that all three agents gain between +42 and +45 pp, with Claude Code moving from 197/511 to 425/511 (38.6% $\rightarrow$ 83.2%, +44.6 pp), Codex from 194/511 to 419/511 (+44.0 pp), and OpenHands+GPT-5.2 from 173/511 to 388/511 (+42.1 pp). The Δ is largest where the no-feedback baseline is highest, since stronger baselines have more localization-stage failures that the Verilator log directly converts into guided edits, while OpenHands’s lower baseline gains slightly less because more of its residual is HDL-semantic rather than localization. Table 5(b) shows that on Claude’s 314 failures the rescue rate is monotone across the three pre-feedback failure stages: 77.7% for *No Edit* (73/94), 73.6% for *Localization Failures* (109/148), and 69.4% for *Repair Failures* (50/72). The pattern holds across design, verification, physical-implementation, and docs categories with one exception: env-and-toolchain failures see only $\sim 60\%$ rescue, because the underlying bug is in tool or simulator semantics and Verilator’s log can be misleading or empty.

Main finding: *Letting the agent read the verifier’s error log helps far more than telling it which file to edit, because the log says where the bug is and what the fix has to look like.*

6 Conclusion

We introduced **Phoenix-bench**, an execution-grounded hardware-issue-resolution benchmark of 511 verified Verilator instances from 114 GitHub repositories, and ran a uniform evaluation of four deployed product agents and eight open-source agentic structures across four LLM backbones plus two diagnostic interventions. (i) Software and hardware are fundamentally different engineering tasks: the same agent loses 37 to 58 pp from SWE-bench Verified to Phoenix-bench because hardware bugs propagate across parallel instantiated modules through signal flow, not along a software-style call graph. (ii) Failures concentrate on design control-flow / FSM bugs, verification testbench bugs, and T3 cases that demand cross-hierarchy signal-flow tracking and coordinated multi-file edits. (iii) Localization granularity, not localization itself, is the controllable lever: a perfect file-level oracle adds only +1.4 pp because the agent then breaks files it should not edit, whereas one round of Verilator-log feedback adds +42 to +45 pp because the log says *where* the bug is and *what* the fix has to look like.

References

Baleegh Ahmad, Shailja Thakur, Benjamin Tan, Ramesh Karri, and Hammond Pearce. 2023. Fixing Hardware Security Bugs with Large Language Models. *arXiv preprint arXiv:2302.01215* abs/2302.01215 (2023), 12 pages.

- Anthropic. 2026. Claude Code Overview. <https://docs.anthropic.com/en/docs/claude-code/overview>. Accessed 2026-04-29.
- Christopher Batten, Nathaniel Pinckney, Mingjie Liu, Haoxing Ren, and Bruce Khailany. 2024. Pyhdl-eval: An llm evaluation framework for hardware design using python-embedded dsls. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*. ACM, New York, NY, USA, 1–17.
- Wenji Fang, Mengming Li, Min Li, Zhiyuan Yan, Shang Liu, Hongce Zhang, and Zhiyao Xie. 2024. Assertllm: Generating and evaluating hardware verification assertions from design specifications via multi-llms. *arXiv preprint arXiv:2402.00386* (2024).
- Weimin Fu, Shijie Li, Yier Jin, and Xiaolong Guo. 2025. HWFixBench: Benchmarking Tools for Hardware Understanding and Fault Repair. In *Proceedings of the Great Lakes Symposium on VLSI 2025*. ACM, New York, NY, USA, 427–434.
- Paul Gauthier. 2024. Aider: AI pair programming in your terminal. <https://aider.chat>.
- GitHub. 2026. About GitHub Copilot Coding Agent. <https://docs.github.com/en/copilot/concepts/about-assigning-tasks-to-copilot>. Accessed 2026-04-29.
- Google. 2026. Gemini CLI. <https://github.com/google-gemini/gemini-cli>. Accessed 2026-04-29.
- Google DeepMind. 2025. Gemini. <https://deepmind.google/models/gemini/pro/>.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. SWE-Bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* abs/2310.06770 (2023), 23 pages.
- Farzaneh Rabiei Kashanaki, Mark Zakharov, and Jose Renau. 2024. HDLEval benchmarking LLMs for multiple HDLs. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, IEEE, San Francisco, CA, USA, 1–5.
- Lingxi Team. 2025. Lingxi: Open-Source Multi-Agent Framework for Repository-Level Issue Resolution. <https://github.com/lingxi-agent/Lingxi>.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. 2025. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556* abs/2512.02556 (2025), 18 pages.
- Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, IEEE, San Francisco, CA, USA, 1–8.
- Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. RtlIm: An open-source benchmark for design rtl generation with large language model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, IEEE, Incheon, South Korea, 722–727.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. 2025. ExpeRepair: Dual-Memory Enhanced LLM-based Repository-Level Program Repair. *arXiv preprint arXiv:2506.10484* abs/2506.10484 (2025), 15 pages.
- OpenAI. 2025. GPT 5.2 System Card. https://cdn.openai.com/pdf/3a4153c8-c748-4b71-8e31-aecbde944f8d/oai_5_2_system-card.pdf.
- OpenAI. 2026. Codex. <https://openai.com/codex>. Accessed 2026-04-29.
- SWE-agent Team. 2025. mini-SWE-agent: The 100-Line AI Agent That Solves GitHub Issues. <https://github.com/SWE-agent/mini-swe-agent>.
- Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>

- Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. Verigen: A large language model for verilog code generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 1–31.
- Yun-Da Tsai, Mingjie Liu, and Haoxing Ren. 2024. RTLFixer: Automatically Fixing RTL Syntax Errors with Large Language Models. In *2024 61st ACM/IEEE Design Automation Conference (DAC)*. IEEE, IEEE, San Francisco, CA, USA, 1–6.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. 2024. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741* abs/2407.16741 (2024), 23 pages.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489* abs/2407.01489 (2024), 17 pages.
- Boyang Yang, Jiadong Ren, Shunfu Jin, Yang Liu, Feng Liu, Bach Le, and Haoye Tian. 2025. Enhancing repository-level software repair via repository-aware knowledge graphs. *arXiv preprint arXiv:2503.21710* abs/2503.21710 (2025), 12 pages.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.
- Bingkun Yao, Ning Wang, Jie Zhou, Xi Wang, Hong Gao, Zhe Jiang, and Nan Guan. 2025b. Location is Key: Leveraging LLM for Functional Bug Localization in Verilog Design. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, IEEE, San Francisco, CA, USA, 1–7.
- Xufeng Yao, Haoyang Li, Tsz Ho Chan, Wenyi Xiao, Mingxuan Yuan, Yu Huang, Lei Chen, and Bei Yu. 2025a. Hdldebugger: Streamlining hdl debugging with large language models. *ACM Transactions on Design Automation of Electronic Systems* 30, 6 (2025), 1–26.
- Jie Zhou, Youshu Ji, Ning Wang, Yuchen Hu, Xinyao Jiao, Bingkun Yao, Xinwei Fang, Shuai Zhao, Nan Guan, and Zhe Jiang. 2025. Insights from rights and wrongs: A large language model for solving assertion failures in rtl design. *arXiv preprint arXiv:2503.04057* abs/2503.04057 (2025), 10 pages.
- Qingyun Zou, Jiahao Cui, Nuo Chen, Bingsheng He, and Weng-Fai Wong. 2026. MHRC-Bench: A Multilingual Hardware Repository-Level Code Completion Benchmark. *arXiv preprint arXiv:2601.03708* abs/2601.03708 (2026), 12 pages.

A Patch Complexity Classification Rules

Coarse difficulty distribution. A simpler, file-and-line-based stratification of the 511 synchronized instances groups gold patches into three difficulty bands: *Easy* (one file and at most 10 edited lines, 100 cases averaging 1.0 files and 4.1 edited lines), *Medium* (two to three files or 11 to 50 edited lines, 113 cases averaging 1.8 files and 19.3 edited lines), and *Hard* (at least four files or more than 50 edited lines, 309 cases). The Hard split is heavy-tailed: it averages 29.6 files and 5,912.6 edited lines, with a median of 211 edited lines because some repository updates include very large generated or vendored diffs. Overall, 80.8% of instances require multi-file or substantial edits, confirming that Phoenix-bench is not dominated by trivial single-line fixes.

To stratify the 511-case Phoenix-bench corpus by developer-patch structure (§5.3, tiers T1/T2/T3), we apply the following rule set to each gold patch. The rules operate purely on the unified diff and consider only HDL source files; scripts, Makefiles, testbenches, and documentation are ignored.

Inputs considered.

- **File filter.** Only files whose extension is one of `.v`, `.sv`, `.vh`, or `.svh` are counted. All other paths in the diff are skipped.
- **Hunk count.** Each unified-diff hunk header (`@@ -... +... @@`) within an HDL file contributes one to the hunk count. Hunks are not merged across line gaps; this matches the raw `git diff` segmentation.
- **File count.** The number of distinct HDL file paths appearing under `diff -git` entries.
- **Hierarchy.** The *top-level directory* of each HDL path (`Path.parts[0]`). A patch is *cross-hierarchy* when it touches files in more than one top-level directory.

Tier rules (evaluated in order).

1. If total HDL hunks ≥ 6 : **T3 Hard**.
2. Else if HDL files > 1 *and* cross-hierarchy: **T3 Hard**.
3. Else if HDL files > 1 (same top-level directory): **T3 Hard** (multi-file).
4. Else if a single HDL file with exactly 1 hunk: **T1 Easy**.
5. Else if a single HDL file with 2 to 5 hunks: **T2 Medium**.
6. Otherwise: T3 Hard (fallback; not exercised in the released corpus).

Resulting distribution. Applying these rules to the 511 verified instances yields **T1: 128 (25.0%)**, **T2: 91 (17.8%)**, and **T3: 292 (57.1%)**. The two patches in the curated set without a matching Verilator snapshot (FPGA-MAFIA_fpga_mafia_pr415, UTOSS_risc-v_pr35) are excluded.

Notes and caveats. The rules deliberately use the raw `@@` hunk segmentation rather than a semantic block merge: a developer change that spans a declaration and its later use will appear as two hunks even when it is a single logical edit, which inflates T2/T3 relative to a "logical block" count. Likewise, any patch that touches more than one HDL file is promoted to T3, even when the second file is a tightly coupled header (`.svh`) of the same module. These choices keep the classifier deterministic and auditable from the diff alone, but they make the released distribution skew more conservative (more T3) than a semantically merged count would.

B Limitation

C Case study: hpdcache_pr33

Figure 10 illustrates a Phoenix-bench case from HPDcache that concretizes the hierarchy / parameter failure category discussed in §5.2. The issue requires wiring a configuration signal through three files in the module hierarchy: `hpdcache.sv` to `ctrl.sv` to `ctrl_pe.sv`. A correct repair extracts the signal from the issue, traces the propagation chain across modules, and applies one coordinated edit. Keyword search alone can find the downstream consumer while missing upstream modules where the

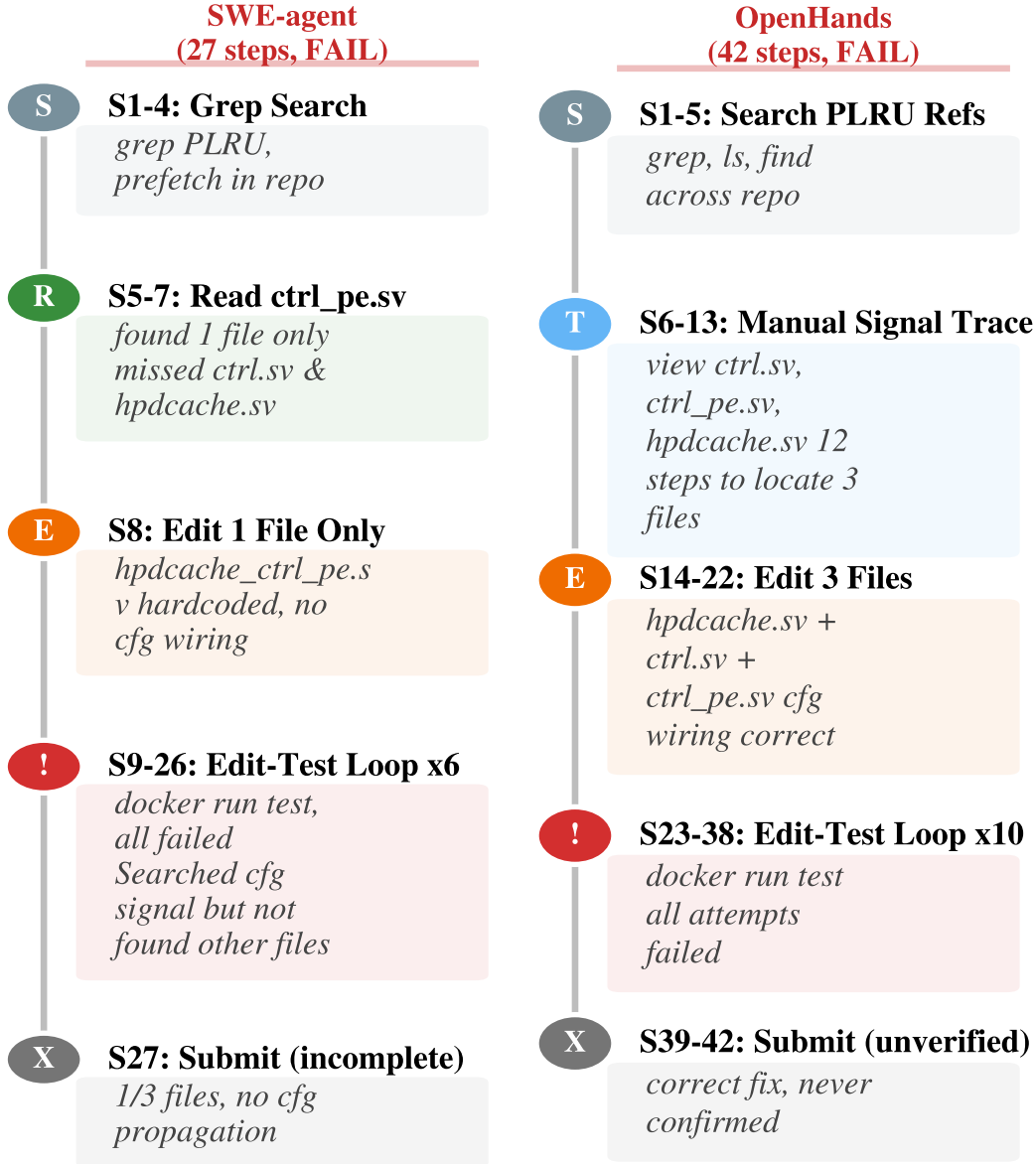


Figure 10: Case study on hpdcache_pr33: a cross-module signal-propagation issue that requires wiring `cfg_prefetch_updt_plru` through the hierarchy.

new signal is not yet mentioned. Even oracle file localization (§5.4) is insufficient here because the agent must still construct the port chain rather than merely identify the file set. This case demonstrates why realistic hardware issue resolution requires understanding signal-flow dependencies, not only finding lexical matches.

D Artifact, Licensing, and Reproducibility Package

The supplementary artifact contains the scripts and manifests needed to audit and reproduce the benchmark runs reported in Tables 3–5. The package includes the benchmark package, Docker build files, executable runners, complete reproduction commands with seeds, budgets, and baseline commit

hashes, the minimal Phoenix-Ref implementation, and the CI workflow. It is organized around the following auditable records.

Repository manifest. The artifact includes a source manifest for the 114 mined GitHub repositories. For each benchmark instance, the manifest records the anonymized repository identifier, upstream repository URL, issue or pull-request identifier, buggy source snapshot commit, developer-patch commit, baseline-agent commit when a baseline implementation is used, detected upstream license, and the location where attribution and NOTICE material are preserved in the released package. Phoenix-bench’s own dataset and runner license is stated at the package root, while each redistributed source snapshot remains governed by its upstream license.

Artifact layout and evaluation items. The released package mirrors the internal benchmark layout used for the experiments: `issues/` stores the issue metadata and prompt-facing problem statement, `snapshots/` stores the pinned repository source snapshots, `patches/` stores the developer patches, and `testbench_verilator/` stores the executable runners and Verilator testbenches. Each evaluation item links one issue record, one source snapshot, one developer patch, and one runner directory. The runner directory normally contains two audit logs: a *buggy log*, which records the compile or runtime error observed before the fix, and a *pass log*, which records the same verifier after the fix passes. These logs are archived with the runner so that the fail-to-pass label can be checked without relying on paper-level summaries.

Docker environments. We provide Dockerfiles and image summaries for the three-layer execution environment used throughout the paper: a base EDA-toolchain image, a repository-dependency image, and an instance-snapshot image. The summaries record the pinned versions and build commands for Verilator, Icarus Verilog, Surelog, and Yosys, together with the per-instance runner entry point used for fail-to-pass and pass-to-pass verification.

Reproduction scripts. The artifact includes the exact reproduction entry points for Tables 3, 4, and 5. Each script records the agent or product interface, model or backbone, prompt template, random seed where applicable, token budget, wall-clock budget, EDA-call budget, baseline implementation commit hash, final verifier command, and aggregation command. For product agents whose proprietary orchestration cannot be replayed exactly, the artifact archives the submitted patches, logs, metadata, and verifier outputs, and the reproduction script recomputes the table rows from those archived run artifacts.

Testbench-feedback disclosure. For the feedback condition in Table 5, the agent receives only the failing compile or runtime log produced by the verifier, together with the executed command and exit status. The feedback does not include the developer patch, the fixed-phase pass log, the gold file list, hidden testcase rationale, or any manually written explanation of the repository. The supplementary scripts include the diagnostic slices used to audit this effect: compile-log-only versus runtime-log-only feedback, full logs versus masked signal/width/module details, one feedback round versus additional rounds, and a no-feedback baseline. For each slice we record whether feedback changes a pre-feedback failure stage (*No Edit*, localization failure, or repair failure) into a correct patch without adding any extra codebase rationale, and we retain qualitative examples where feedback still fails.

One representative case is `testbench/pulp-platform_axi_pr138/compile.log`. The failing compile log directly names the unresolved cross-module token, the testbench line, the originating module, and the source expression:

```
Error-[XMRE] Cross-module reference resolution error
tb_axi_xbar_cfg_localparam_patch.sv, 308
token 'cfg_NoMstPorts'. Originating module
'tb_axi_xbar_cfg_localparam_patch'.
Source info: assign cfg_probe = i_axi_xbar.cfg_NoMstPorts;
```

This is exactly the kind of information that explains the large gain from a single feedback round: the log turns a broad repository search problem into a specific hierarchy/localparam repair target. The corresponding pass log records only the successful verifier outcome (PASS: `axi_xbar.sv` uses `cfg_NoMstPorts localparam.`), so the pass phase is archived for audit but not shown to the agent during the feedback intervention.

Reference implementation and CI. The release includes a minimal Phoenix-Ref implementation that reads a benchmark instance, checks out the pinned snapshot, applies a candidate patch, invokes the same Docker verifier used for the tables, and emits the standardized `result.json` consumed by the aggregation scripts. We also provide a continuous-integration workflow that builds the base image, validates the manifest schema and per-instance checksums, runs a smoke subset of the fail-to-pass and pass-to-pass runners, replays the Phoenix-Ref example, and checks that the table-generation scripts can parse archived run artifacts. This CI job is intended as the minimum independent audit path for the released benchmark package.

Integrity checks. Each instance is accompanied by checksums over the issue metadata, repository snapshot, developer patch, fail-to-pass and pass-to-pass testbenches, runner configuration, and expected verifier output. The release includes a validation script that recomputes these checksums before running any benchmark job, so an independent evaluator can confirm that the reproduced tables were generated from the same instance set used in this paper.

Fail-to-pass runner example. The artifact also includes concrete per-instance runners; for example, `testbench_verilator/aolofsson_oh_pr74/run_verilator.sh` documents the full fail-to-pass protocol for one OH/GPIO issue. The runner copies the pinned repository snapshot into a writable scratch directory, applies the developer patch in reverse to construct the BUGGY phase, and then executes the native smoke check plus a Verilator testbench. It then recreates the scratch directory, applies the developer patch forward to construct the FIXED phase, and reruns the same checks. This instance targets `src/common/hdl/oh_tristate.v` and `tb_oh_tristate_verilator.sv`: before the patch, Verilator reports `%Error-MODMISSING` because the testbench instantiates `oh_tristate` but the module is absent; after the patch, the testbench drives the bidirectional `io` port with four external values and emits only `PASS:` lines when `in` reflects those values. The runner accepts the instance only when the buggy phase has nonzero exit status and the fixed phase exits zero, after which it reports the minimum testcase count and Verilator coverage. This example shows the same pattern used by Phoenix-bench at scale: the benchmark records the exact repository snapshot, patch, testbench, runner command, and logs, and the resolved-rate label is derived from deterministic EDA execution rather than manual or LLM judgment.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes].

Justification: The abstract and introduction state the paper’s scope as a practice-driven benchmark suite plus a reference implementation, and the empirical claims are supported in Sections 3 and 4.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes].

Justification: The Conclusion and Limitations section discusses language scope, EDA-tool dependence, benchmark size, and potential public-repository contamination.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A].

Justification: The paper introduces a benchmark suite and empirical evaluation, not theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: Sections 3–4 describe the data collection funnel, verification environment, evaluated systems, models, metrics, and reference implementation interfaces; Appendix D further documents the released manifests, Docker environments, Phoenix-Ref implementation, CI workflow, checksums, run scripts, seeds, budgets, and baseline commit hashes used to reproduce the reported tables.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes].

Justification: Appendix D describes the supplementary artifact, which includes the benchmark manifest, three-layer Docker environment, minimal Phoenix-Ref implementation, CI workflow, per-instance checksums, and reproduction scripts for Tables 3–5, including seeds, budgets, baseline commit hashes, verifier commands, and aggregation commands.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.

- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#).

Justification: The paper has no training procedure; the experimental section specifies benchmark instances, models, systems, EDA execution environment, and evaluation metrics.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[No\]](#).

Justification: The reported results are deterministic pass/fail rates over a fixed benchmark suite; the current draft does not include confidence intervals or bootstrap estimates.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The authors should answer [\[Yes\]](#) if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [\[No\]](#).

Justification: The paper specifies the EDA tools and Docker setting, but it does not yet provide full wall-clock time, CPU/memory, or total compute cost for every experiment.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes].

Justification: The work studies public hardware repositories and LLM-agent evaluation without human-subject experiments or deployment on users.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No].

Justification: The current draft frames the benchmark's research utility and limitations, but it does not yet include a dedicated broader-impact discussion covering both benefits and misuse risks.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A].

Justification: The paper does not release a pretrained model or high-risk generative asset; the benchmark is derived from public hardware repositories and does not include private user data.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes].

Justification: Appendix D describes the released source manifest, which enumerates the 114 mined GitHub repositories, source snapshot and developer-patch commits, baseline implementation commits where applicable, detected upstream licenses, Phoenix-bench's own license, and the attribution or NOTICE locations preserved in the artifact.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes].

Justification: Phoenix-bench and Phoenix-Ref are documented in Sections 3 and 4, including construction, labels, verification protocol, and framework interfaces.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.

- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A].

Justification: The work does not involve crowdsourcing or human-subject experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A].

Justification: The work does not involve crowdsourcing or human-subject experiments.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes].

Justification: LLMs are central to the evaluated agents and to Phoenix-Ref; the paper describes the model backbones, agent interfaces, and LLM-based components used in the study.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.